# Training Coding Agents Using Later Modification Outcomes

## Background on the Problem

A coding agent can produce an implementation that passes the current tests, but leaves the codebase in a worse state for the next agent. For example, a file-backed service may correctly save individual records but scatter validation and persistence across several functions, making a later transaction feature harder to implement without breaking existing behavior. We propose training the initial code writer using the outcomes of subsequent modifications. The aim is to generate code that works now and supports later changes by an editing agent.

This problem has empirical support, although its severity depends on the model and task. CodeThread reports downstream resolve-rate drops of up to 13.1 percentage points when agents build on agent-written rather than human-written code, and finds that static maintainability measures do not fully explain the differences [2]. SlopCodeBench found the same pattern when agents repeatedly extend their own code, with no agent fully solving any problem and structural erosion rising in 77% of trajectories and verbosity in 75.5% [10]. Our project will test whether these downstream outcomes provide a useful training signal for the initial writer.

## Related Literature

Research has moved beyond evaluating a single correct implementation. MaintainBench applies changing requirements to programming tasks, while its MaintainCoder system improves generation through an orchestrated workflow for requirement analysis, design, implementation, and refinement [1]. CodeThread controls the initial authorship while holding the follow-on task fixed [2]. SWE-CI evaluates maintenance through repeated continuous-integration cycles [3], and SlopCodeBench measures how code quality degrades as agents extend their own solutions across successive changes [10]. Together, these works establish that later modification performance can be measured. MaintainCoder also already uses a short downstream probe to estimate maintenance effort, so neither such a probe nor a new sequential benchmark is our proposed novelty.

Training for code quality is also established. ReaL combines current unit-test results with program-analysis feedback to train for correctness and code quality [4]. Sivapiran and Uddin study DPO and BoNBoN alignment for both functional and nonfunctional code quality [5]. These findings motivate strong training baselines: an improvement over an untrained writer alone would not show that later-edit feedback adds value. The specific comparison we need is against learning from current correctness and from static quality signals, using comparable data and training budgets.

The broad training concept has a close public precedent. CodeRhapsody proposes generating initially correct alternatives, introducing later requirements, and using their maintenance trajectories as training feedback [6]. It presents this as a proposal rather than a reported training experiment. We therefore scope our contribution as a controlled implementation and evaluation of this idea: preferences between initially passing programs will come from repeated, executable later-edit outcomes, and only the initial writer will be updated. We will test transfer to unseen task families and change types, with independent-editor evaluation as a stretch goal. This focuses the project on whether the training signal works beyond the setup that generated it.

## Project Goals and Scope

Our research question is whether training a code writer on the success of later edits or additions helps the writer produce code that is easier to modify, compared with training it solely on the success of the initial code. We expect that it will lead to more successful changes without making the first version of the code worse. We will test this with one small open-weight Python code writer, one fixed editor, and short sequences of changes. Testing with a second editor and on real repository tasks is a stretch goal, and we will not claim that our results show how easy the code is for human developers to maintain.

## Approach

We will build a pipeline that generates several working versions of a program, tests how easily another agent can modify them, and uses those results to train the original code writer. The editing agent will remain unchanged so that we can measure the effect of training the writer.

**Task construction.** We will target 60 Python task families: 36 for training, 8 for development, and 16 for evaluation. Each task will include an initial programming request, later changes, and tests for the required behavior. Tasks will involve interacting components, such as validation and storage, while leaving the internal design up to the writer.

We will use team-authored tasks and adaptations of permissively licensed examples, drawing on MaintainBench and CodeThread for task design. Each task's description and tests will be checked to make sure the correct solutions pass and deliberately wrong solutions fail. Tasks that are close variants of each other will stay in the same split, so the model never trains on something it is later tested on. Training will use sequences of two changes. Testing will use sequences of three changes and new kinds of changes the writer has not seen.

**Initial implementations.** For each training task, we will generate up to eight implementations. The writer will see only the initial request and public checks. Only implementations that pass the initial correctness tests will enter the comparison, and identical copies will be removed.

**Example.** Two record-storage programs might both correctly support adding, updating, and deleting records. We then ask the editing agent to add conflict detection. We compare which starting program allows these features to be added more reliably while preserving existing behavior.

**Editing protocol.** Each implementation will receive the same later requests and editing budget. The editor may inspect the code, modify it, and run public checks, but it cannot access private evaluation tests. We will select the editor and budget using development tasks, then keep them fixed. Each trial will start from a fresh copy of the original implementation. We will test two different change sequences and repeat each three times.

**Training preferences.** We will score each starting implementation by how many consecutive changes the editor successfully completes before its first failure. Success requires passing both the new-feature tests and tests for existing behavior. Implementations with consistently higher scores will become the preferred training examples. Ties and inconsistent results will receive no preference label. Time, token usage, and the amount of code changed will be reported separately.

## Algorithms and Implementation

We will fine-tune the initial code writer using Direct Preference Optimization (DPO) [7]. Each training example will contain the initial programming request and two implementations that both pass the initial correctness tests. The preferred implementation will be the one that led to more reliable later modifications. DPO trains the writer to favor that implementation. The writer's training input will contain only the initial request and the two original implementations. Future requests, modified code, and private tests will remain hidden. Their outcomes determine the preference labels.

We will implement DPO with Hugging Face TRL and use QLoRA to reduce GPU memory requirements [8]. QLoRA trains a small set of additional parameters called adapters while keeping the original model weights frozen and stored in a compressed format. Training settings will be selected using development tasks before evaluating on held-out tasks. We will balance examples across task families so that a few tasks do not dominate training.

We will target 100–300 preference pairs, with a fixed limit on the number of implementations generated per task. We will report how many comparisons produce usable preferences and how consistently repeated editing trials agree. Team members will review a sample of preferred implementations to check that their behavior matches the specification.

## Baselines and Ablations

We will compare the same starting writer under four controls: its original checkpoint; a maintainability-focused prompt; correctness-only fine-tuning on initially passing examples; and DPO using static-quality rankings among initially passing candidates. The static ranking will use a development-frozen Radon maintainability score. Training controls will share the source task pool, adapter capacity, and comparable training-token budgets. Inference controls will share the same editor and editing budgets. Any initial-generation cost differences will be

reported. A small MaintainCoder-style workflow comparison is a stretch goal, not a substitute for the training controls.

Ablations will compare preferences from one change against two changes, and from one repetition against three. This test whether longer probes or repeated outcomes add useful information. We will also compare a size-matched subset of initially passing programs and test with an independent editor where feasible, to examine whether gains rely on code length or conventions specific to the training editor. We will also repeat the held-out evaluation with comments and docstrings removed from every program before the editor sees it, and we will report comment density for each method, to test whether gains depend on comments that act as hints for the editor.

## Hardware, Software Platform, and Language

We will implement the project in Python 3.11 using PyTorch, Transformers, TRL, PEFT, bitsandbytes, pytest, Hypothesis, and Radon. The planned writer is Qwen2.5-Coder-7B-Instruct, a decoder-only transformer with approximately 7.6 billion parameters [9]. We will evaluate Qwen2.5-Coder-14B-Instruct as the initial editor candidate [11]. The final editor checkpoint, quantization, prompts, tools, and budgets will be selected on development tasks and frozen before collecting training preferences.

Training and inference will run sequentially on an RTX 5090 with 32 GB GPU memory and 64 GB system RAM, using Linux or WSL2. CPU component tests will run in GitHub Actions, and generated programs will execute in resource-limited Linux containers. We may also use other GPUs that we personally own for training and inference.

**Compute estimate:** Our training-data collection has an upper bound of 3,456 editing checkpoints: 36 tasks x 8 candidates x 2 sequences x 3 repetitions x 2 changes. The principal evaluation adds up to 3,168 checkpoints, assuming one fresh program per held-out task and evaluated configuration, with three seeds for each trained method. These totals might be reduced by initial failures, duplicate candidates, or stopped trajectories.

At an illustrative average of one minute per checkpoint, these main editing runs would require approximately 110 GPU-hours. This is an estimate and excludes initial generation, development calibration, and adapter training. Development profiling will determine the final runtime budget and sequence-length limits. In addition, ablations and stretch evaluations will need to be budgeted separately.

## Validation Methods

**Component tests.** We will use pytest to test each substantial module with fixed inputs and known outcomes:

- **Task registry and prompts:** verify correct dataset splits and ensure future requests never appear in the initial writer's prompt.
- **Artifact parser:** correctly extract multiple files, preserve source bytes, and reject ambiguous filenames or unsafe paths.
- **Editing runner:** correctly handle successful execution, errors, timeouts, and process cleanup.
- **Contract evaluator:** accept reference implementations and reject planted bugs, checking new features and existing behavior separately.
- **Scorer and preference builder:** reproduce hand-calculated scores and exclude ties, inconsistent comparisons, and initially incorrect programs.
- **Training pipeline:** verify the DPO loss, token masking, adapter updates, and unchanged frozen weights.
- **Experiment storage and analysis:** verify source tracking, accurate metrics, and resuming completed trials without repeating model calls.

Separate integration checks will verify sandbox restrictions and exercise the complete pipeline with a deterministic mock model.

**Experimental evaluation.** We will compare all methods on the same held-out tasks and the same editing budgets. Our main score will be the fraction of a short sequence of changes that the editor completes before its first failure. We will also report how often the first version works, how often a full sequence of changes succeeds, how often

old behavior breaks, and how many comments each method's code contains. We will report these for all generated programs and for the subset that worked at first.

To test whether comments explain any gains, we will repeat the held-out evaluation with all comments and docstrings removed before the editor sees the code. If the gains stay the same without comments, they come from the code itself and not from hints written for the editor.

Training will use several random seeds, and each program will be edited several times. We will compare results task by task and calculate confidence intervals by resampling the task families. We will measure cost with token usage and runtime. The evaluation will include tasks and kinds of changes the writer has not seen. A second editor and tests on real repository tasks are stretch goals. When a change fails, we will sort the failure by cause, such as a bug in the first version, a mistake by the editor, an unclear test, or a problem with our infrastructure.

## Deliverables

- A reviewed set of coding tasks with reference solutions, tests, and separate training, development, and held-out splits
- A rerunnable pipeline and tests that generate program versions, run the editor, score each version, and build preference pairs
- A set of better and worse code pairs, with a sample reviewed by our team
- The trained writer, released as adapters for the model used
- Results comparing the trained writer against matched baselines on held-out problems, with ablations on how the pairs are built and whether comments matter
- Measurements of initial correctness, later-change success, regressions, and computing cost, shown in visualizations
- Instructions to reproduce our experiments
- Stretch goals if time allows, such as a second editor, real repository tasks, and a prompting-based comparison

The final report and presentation will describe the research question, methods, results, and limitations. All releasable artifacts will be distributed through the project repository.

## Human Required Tasks

The human required tasks in this project include, but are not limited to:

1. Select realistic change scenarios that are of proper difficulty such that they can actually be trained on.
2. Specify edge cases that are not explicitly determined in the specification for the problem the LLMs are solving and resolve ambiguous requirements.
3. Review the tests and reference solutions before looking at any model results, so a test that rejects valid code or accepts broken code gets fixed before it affects the scores.
4. Work out why a change failed, for example, whether the task was too hard, the first program already had a bug, or the editor made a mistake.
5. Read parts of the winning and losing programs to check whether their comments are hints for the editor, so a program only wins because of its code and not because of its comments.
6. Assess whether improvements generalize, compare findings with previous research, and reach conclusions based on the evidence

## Rough Timeline and Milestones

These are team targets for the remaining semester; final submission and presentation dates follow the course schedule.

| Target | Milestone |
|---|---|
| Oct 9 | Finalize the question, scope, task blueprints, and experimental protocol. |

| Target | Milestone |
|---|---|
| Oct 23 | Complete reviewed contracts, component tests, splits, and calibrated writer-editor setup; freeze collection settings. |
| Nov 6 | Collect and audit preference data; train the first adapter and matched training controls. |
| Nov 20 | Complete the principal held-out evaluation, training seeds, and probe ablations. |
| Dec 4 | Finish generalization checks, error analysis, plots, documentation, and release candidate. |
| End of semester | Submit the final report, presentation, adapters, and reproducible repository. |

GitHub Repo Link: https://github.com/Riya-T/capstone-machine-learning

## References


[1] Wang, Z., et al. (2025). MaintainCoder: Maintainable Code Generation Under Dynamic Requirements. NeurIPS 2025. https://arxiv.org/abs/2503.24260v3

[2] Patel, S., et al. (2026). Is Agent Code Less Maintainable Than Human Code? arXiv preprint, revised September 29. https://arxiv.org/abs/2606.21804v2

[3] Chen, J., et al. (2026). SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration. arXiv preprint. https://arxiv.org/abs/2603.03823

[4] Yao, F., et al. (2025). Training Language Models to Generate Quality Code with Program Analysis Feedback. https://arxiv.org/abs/2505.22704

[5] Sivapiran, S., & Uddin, G. (2026). Reward-Free Code Alignment from Pretrained or Fine-Tuned LLM: Unpacking the Trade-offs for Code Generation. FSE 2026. https://arxiv.org/abs/2606.28998

[6] Cox, B., & CodeRhapsody. (2026). Training Superhuman Software Architects: A Proposal. Public proposal, May. https://www.coderhapsody.ai/docs/superhuman-architecture

[7] Rafailov, R., et al. (2023). Direct Preference Optimization: Your Language Model is Secretly a Reward Model. NeurIPS 2023. https://arxiv.org/abs/2305.18290

[8] Dettmers, T., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023. https://arxiv.org/abs/2305.14314

[9] Qwen Team. Qwen2.5-Coder-7B-Instruct. Official model card. https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct

[10] Orlanski, G., Roy, D., Yun, A., Shin, C., Gu, A., Ge, A., Adila, D., Roberts, N., Sala, F., & Albarghouthi, A. (2026). SlopCodeBench: Benchmarking How Coding Agents Degrade Over Long-Horizon Iterative Tasks. arXiv preprint. https://arxiv.org/abs/2603.24755

[11] Qwen Team. Qwen2.5-Coder-14B-Instruct. Official model card. https://huggingface.co/Qwen/Qwen2.5-Coder-14B-Instruct